



Smart Receipts, Smarter Spending

Demo

1. User Authentication – Login Page

Scenario 1.1: Launching the Application

When users visit the ReceiptCAT web app (<https://d3d2ipjhzxtb9u.cloudfront.net>), they are greeted with the secure login interface powered by Amazon Cognito Hosted UI.

The interface supports both Google federated login and email-based authentication, ensuring flexibility and accessibility across devices.

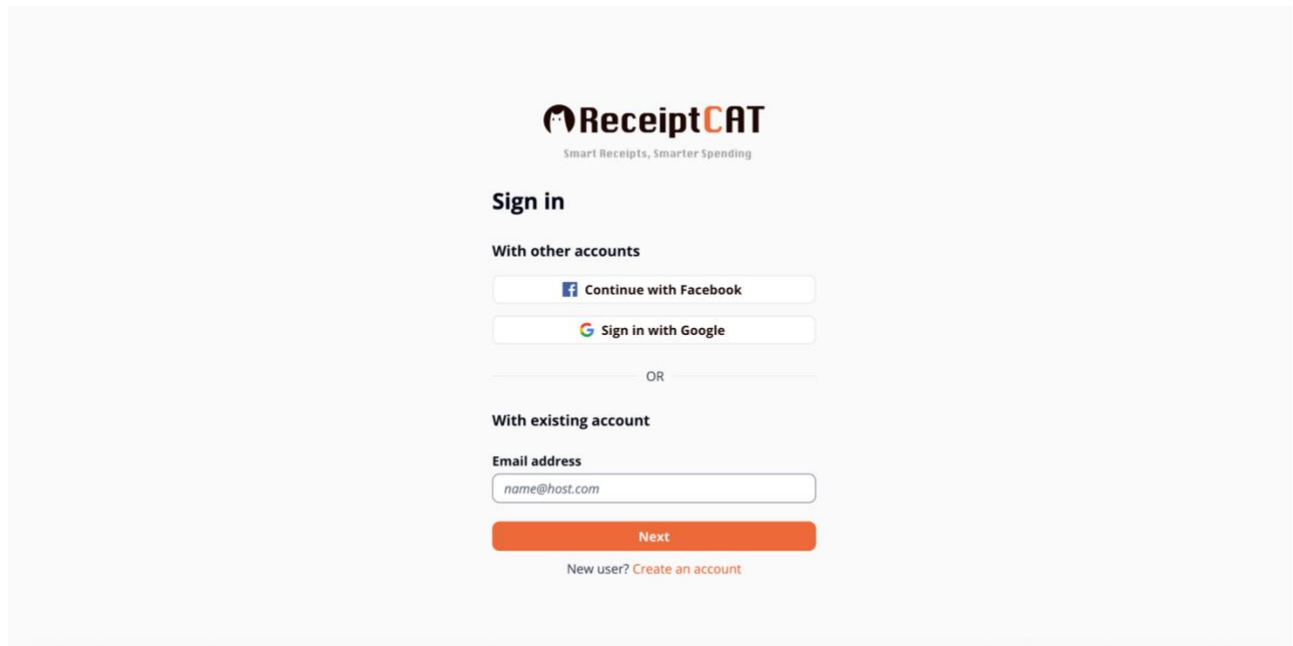


Figure 1 - Login Page

Feature Highlights:

1. Authentication is fully handled by Amazon Cognito.
2. Supports two login methods:
 - (1) Google Sign-In via OAuth2 (federated identity).
 - (2) Email/Password Login with enforced email verification.
3. Upon successful login, Cognito returns secure ID, Access, and Refresh tokens that authorise subsequent API requests.
4. Unverified users cannot proceed until they complete email verification.

Scenario 1.2: Google Login Flow

Users choosing Google Sign-In are redirected to the Google OAuth2 consent screen.

Once authenticated, Cognito automatically associates their Google identity with a unique internal Cognito user ID (sub).

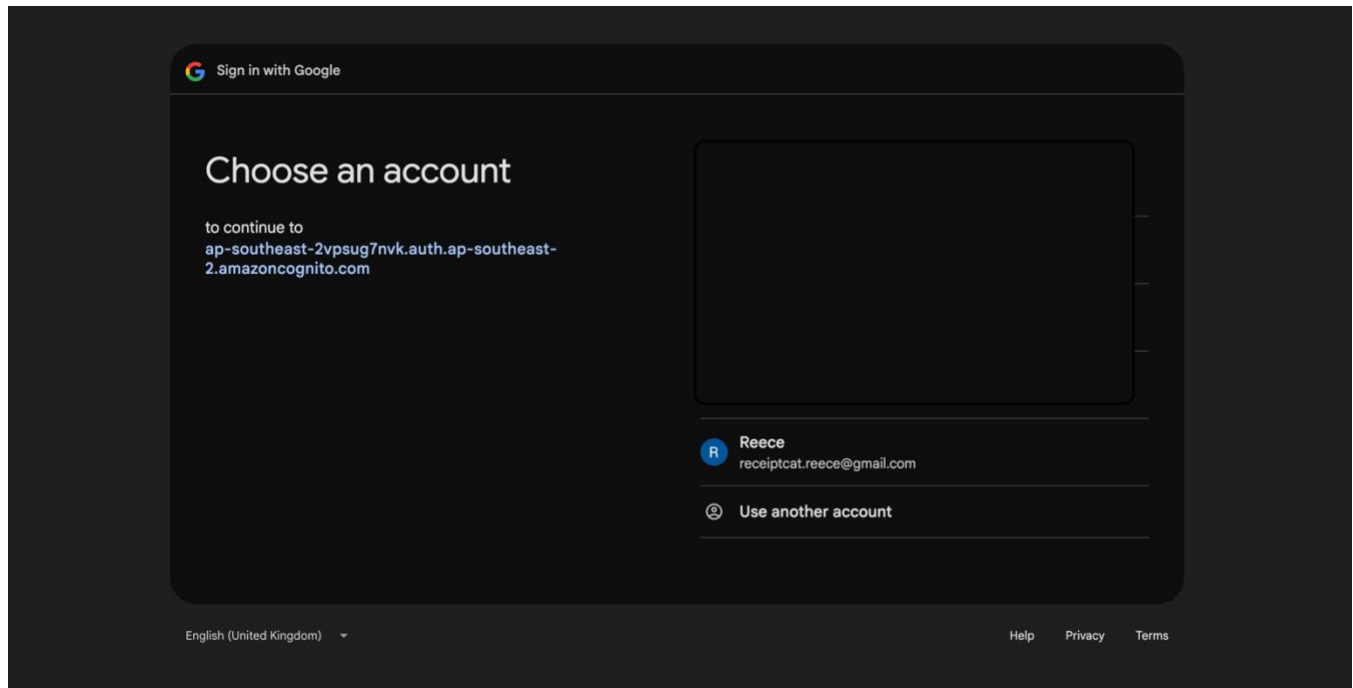


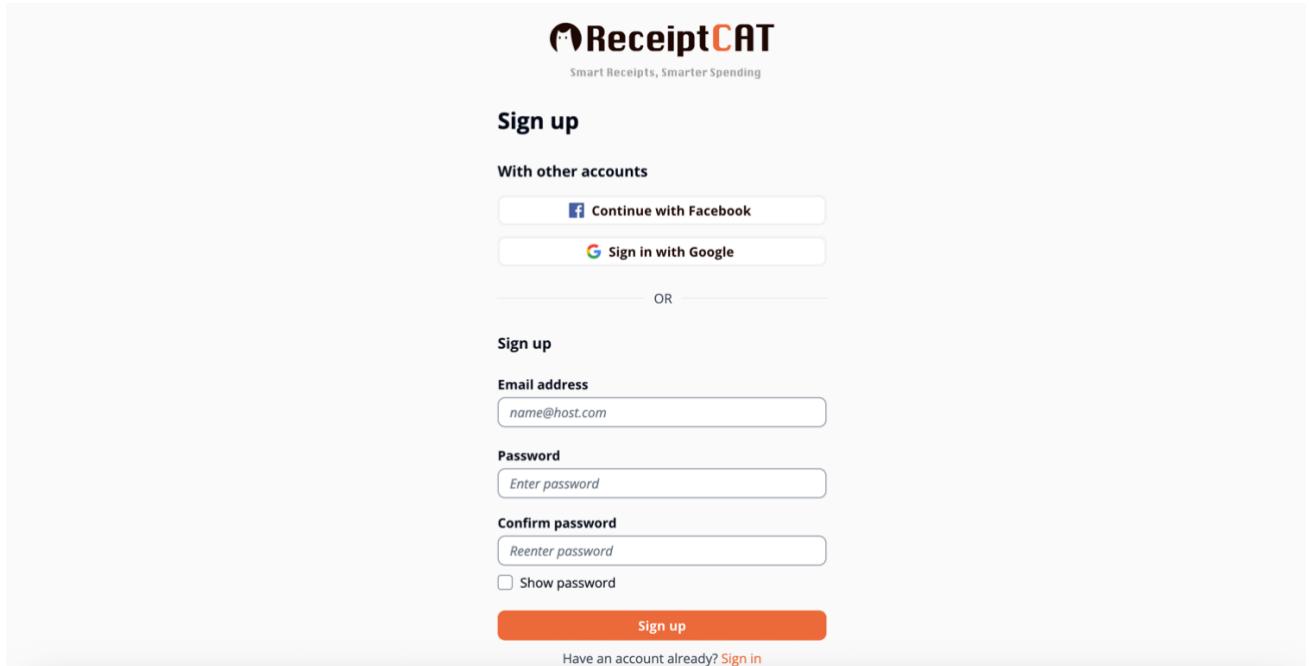
Figure 2 – Countine with Google

Feature Highlights:

1. Fully OAuth2-compliant redirection handled by Cognito.
2. No credentials are processed or stored by the application itself.
3. Tokens are stored securely in sessionStorage for the duration of the session.

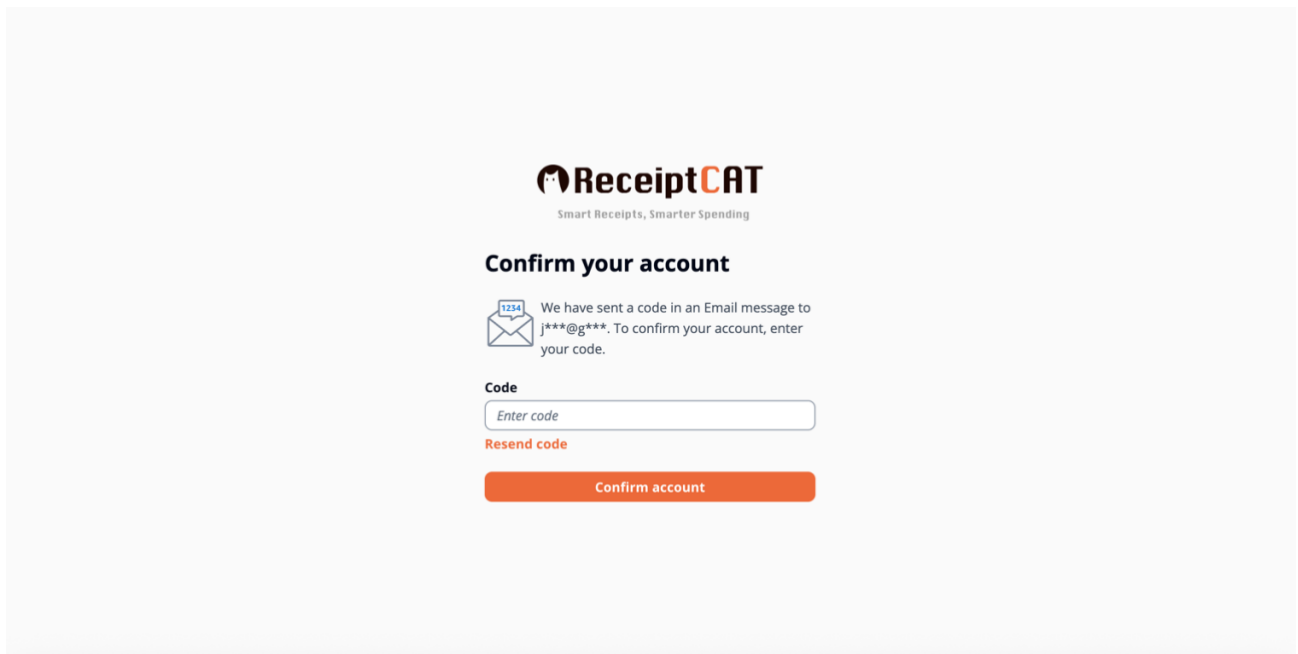
Scenario 1.3: Email Verification (For New Users)

New users who register via email must verify their address using a one-time verification code sent by Cognito. This step ensures that only legitimate, verified accounts gain access to the app.



The screenshot shows the ReceiptCAT sign-up interface. At the top is the ReceiptCAT logo with the tagline "Smart Receipts, Smarter Spending". Below the logo is the heading "Sign up". Underneath, there's a section "With other accounts" featuring two buttons: "Continue with Facebook" and "Sign in with Google". A horizontal line with the word "OR" in the center separates this from the email sign-up section. The email sign-up section has the heading "Sign up" and three input fields: "Email address" (with placeholder "name@host.com"), "Password" (with placeholder "Enter password"), and "Confirm password" (with placeholder "Reenter password"). Below these fields is a checkbox labeled "Show password" and an orange "Sign up" button. At the bottom, there's a link that says "Have an account already? Sign in".

Figure 3 – Sign Up via email



The screenshot shows the ReceiptCAT account confirmation interface. At the top is the ReceiptCAT logo with the tagline "Smart Receipts, Smarter Spending". Below the logo is the heading "Confirm your account". Underneath, there's a message icon with a code "1234" and text: "We have sent a code in an Email message to j***@g***. To confirm your account, enter your code." Below this message is a "Code" input field with placeholder "Enter code". Under the input field is a link labeled "Resend code" in red. At the bottom is an orange "Confirm account" button.

Figure 4 – Email verification

Feature Highlights:

1. Enforced verification before any /app route access.
2. Verification status automatically updated in the Cognito user pool.
3. Secure handling of temporary passwords or codes.

2. Receipt Upload and Preview Editing

Scenario 2.1: Uploading a Receipt (Happy Path)

This scenario demonstrates the standard upload workflow, where a user selects an image file, reviews it in the preview editor, and confirms the upload to complete processing.

The upload interface offers three options: Take a Photo, Upload from Device, and Upload from File. In this demonstration, the Upload from *Device* option is used.

Step 1: File Selection

The user clicks the Upload button on the navigation bar, chooses Upload from Device, and selects an image file (e.g., receipt.jpg).

Once selected, the image is temporarily stored in the browser's sessionStorage to maintain state across navigation.

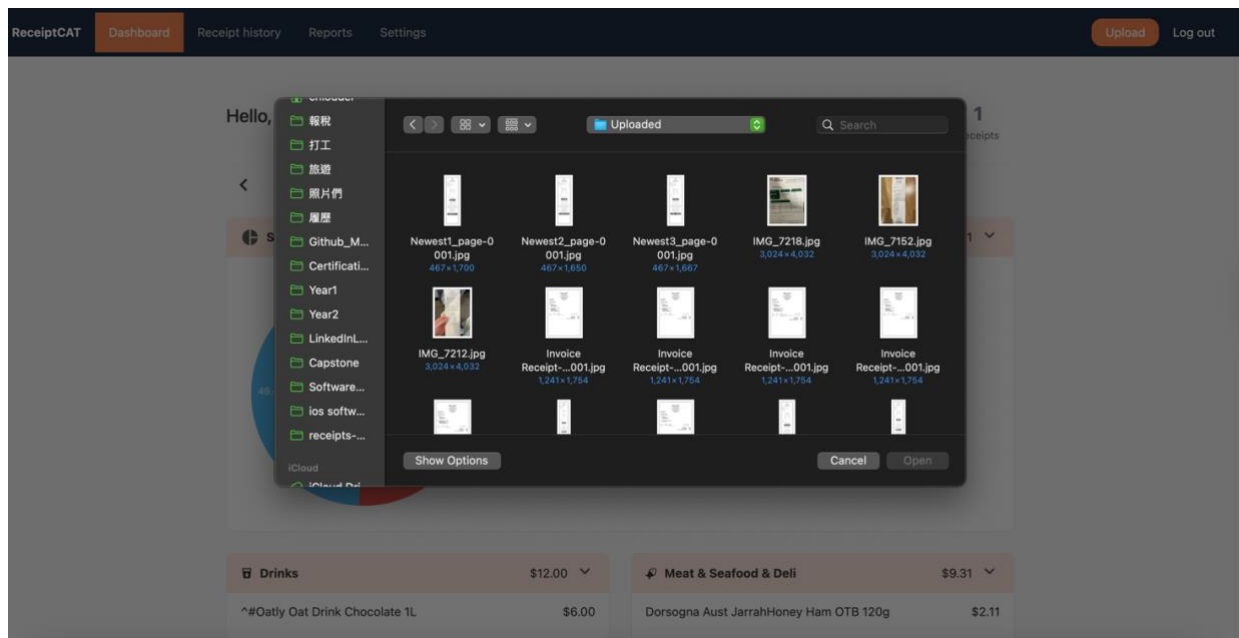


Figure 5 – Upload from device

Step 2: Preview and Edit

The user is redirected to the Preview Page (/app/preview), where the uploaded image is displayed.

Here, users can:

1. Verify that the correct receipt image has been selected.
2. Use the Crop tool to focus on relevant sections.
3. Use the Blackout tool to manually redact sensitive details such as card numbers or personal information.
4. All edits occur locally within the browser, ensuring unredacted data never leaves the user's device.

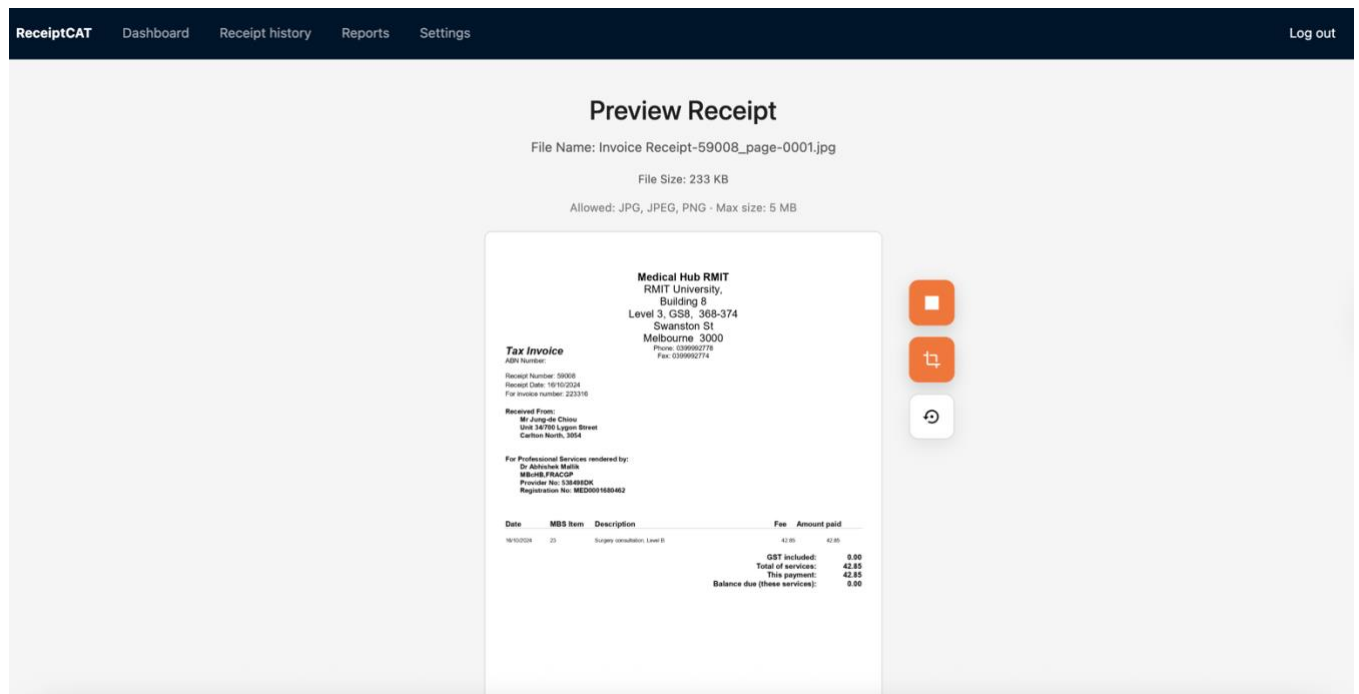


Figure 6 – Preview Page

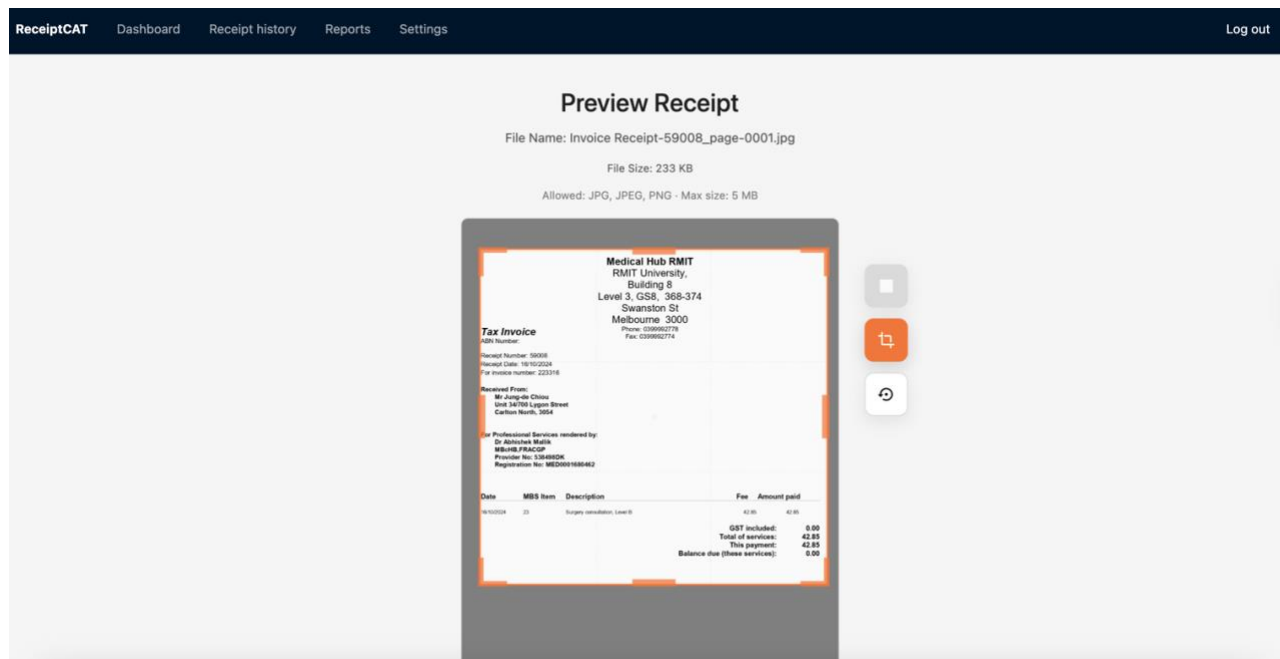


Figure 7 – Cropping tool demonstration

Step 3: Confirm and Upload

After verification and optional edits, the user clicks Confirm Upload. The frontend then retrieves a presigned URL from the backend via the /presign API, uploads the redacted image to the S3 bucket (receiptcat-backend-dev-receipts/tmp/users/{sub}/...), and redirects the user to the Receipt History page upon success.

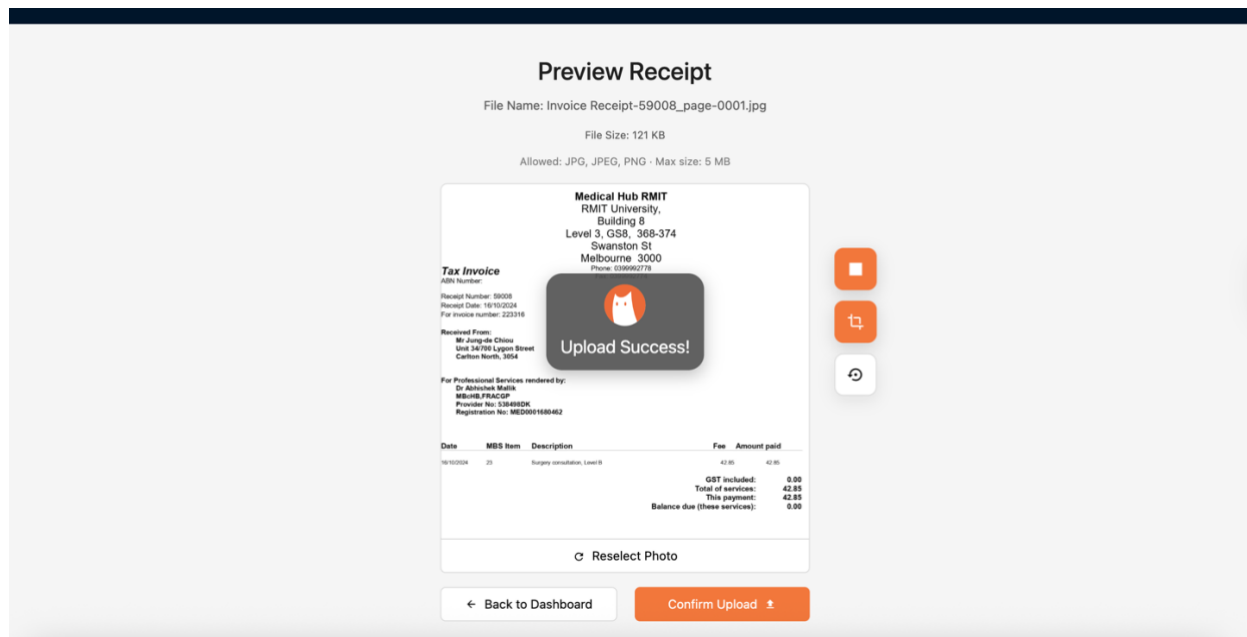


Figure 8 – Success Upload

Feature Highlights:

1. Supports JPG, JPEG, and PNG up to 5 MB.
2. Provides local editing (Crop & Blackout) before upload.
3. Upload handled securely via Cognito-authenticated presigned S3 URL.
4. Automatically redirects to Receipt History after a successful upload.

Scenario 2.2: Invalid File Type or Oversized Upload

If a user attempts to upload an unsupported file format (e.g., PDF or HEIC) or a file exceeding 5 MB, the system performs validation before initiating the upload.

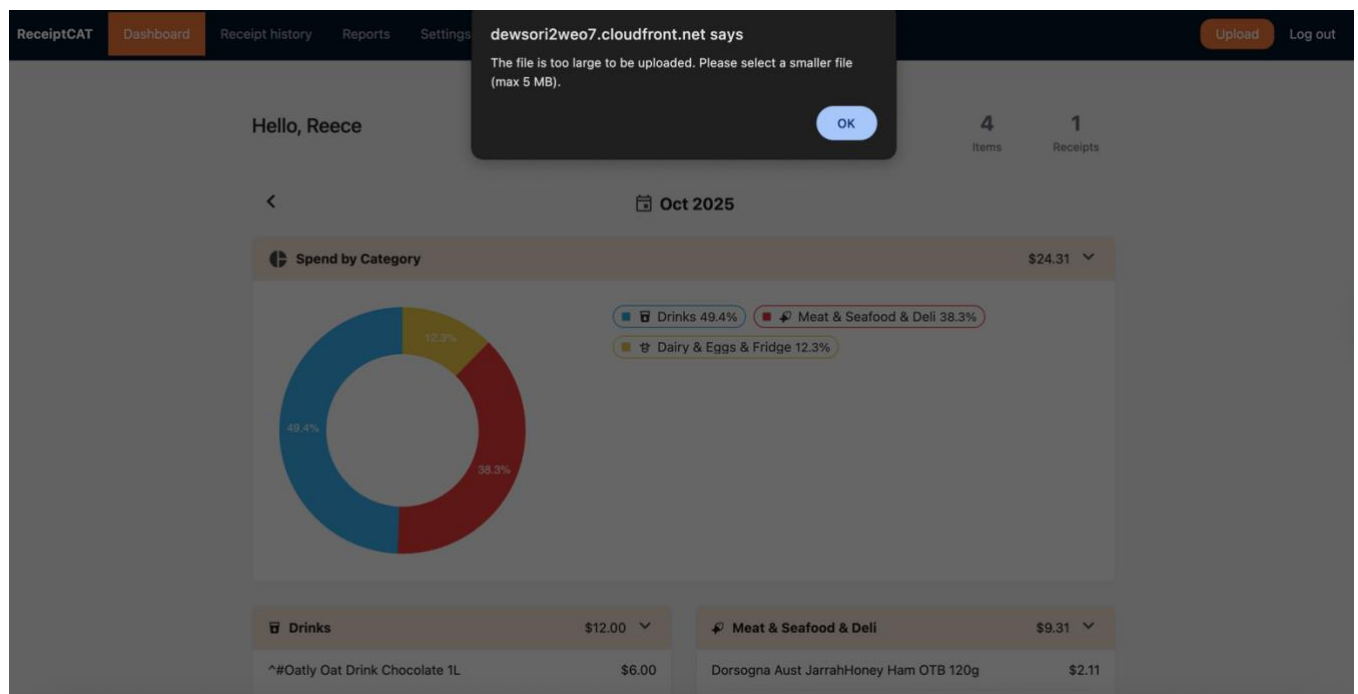


Figure 9 – File size too large

Feature Highlights:

1. Client-side validation prevents unsupported files from being uploaded.
2. The validation checks are performed before sessionStorage updates or API calls, ensuring a consistent UX.
3. Error messages are displayed once detected, allowing the user to retry immediately.

Scenario 2.3: Upload Failure (Network or S3 Error)

If the upload process fails due to a network interruption, expired presigned URL, or S3 error, the client catches the failure and notifies the user without data loss.

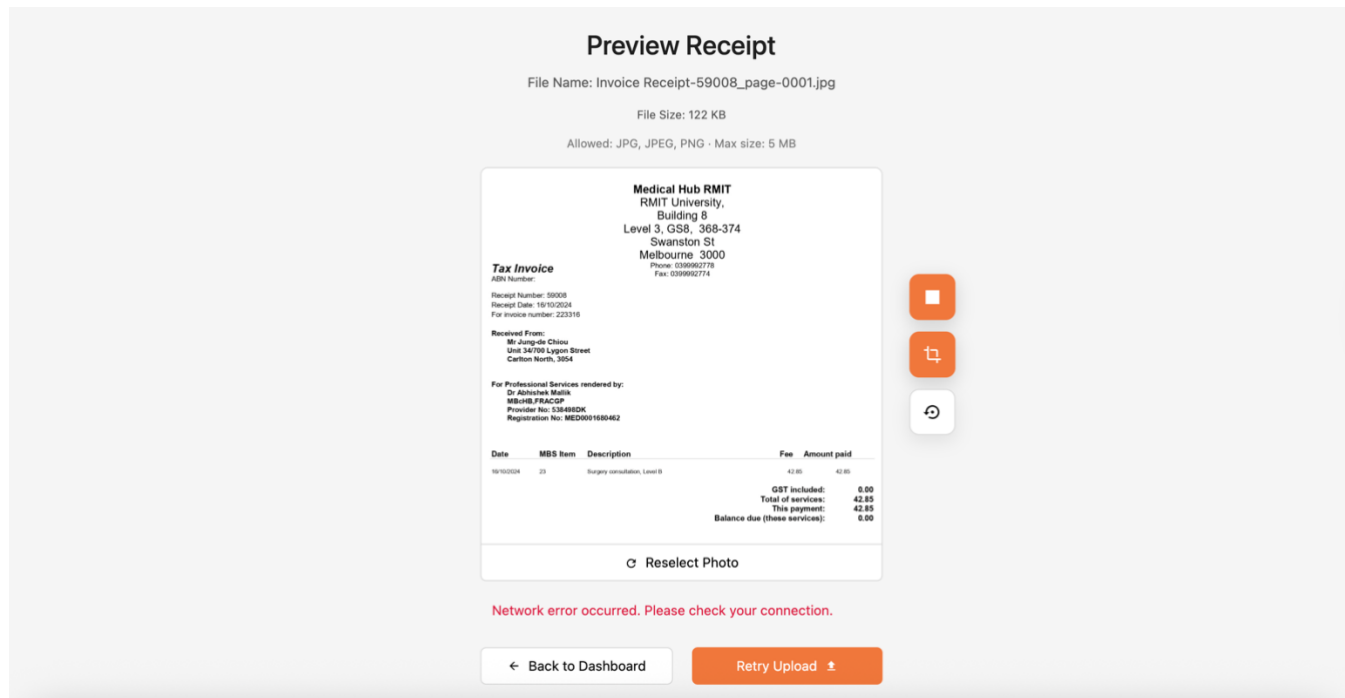


Figure 10 – Upload failure

Feature Highlights:

1. Automatic retry prompt for transient network issues.
2. Secure handling of partial uploads — no incomplete files are retained in S3.
3. Session state is preserved, allowing the user to retry without re-selecting the file.

3. Dashboard – Expense Visualisation and Filtering

Scenario 3.1: Default Monthly Overview

After logging in, users are directed to the Dashboard page (/app), which displays a summary of their spending for the current month by default.

The dashboard aggregates all receipts uploaded within the month. It presents three key topline statistics — Total Spending, Total Items, and Number of Receipts — alongside an interactive pie chart that visualises the breakdown of spending by category.

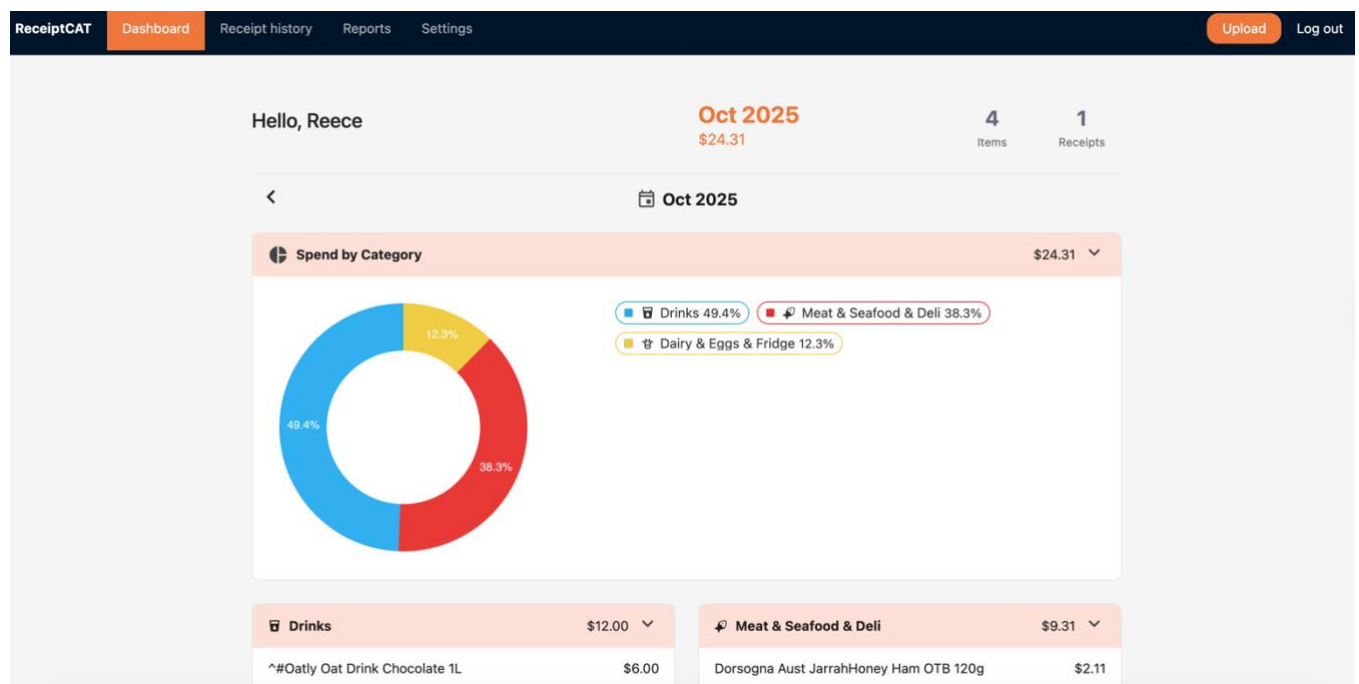


Figure 11 – Dashboard page

Feature Highlights:

1. Automatically loads the current month's date range (e.g., 2025-10-01 to 2025-10-31).
2. Fetches category summary data from the backend API (/users/{userId}/dashboard?fromDate=&toDate=).
3. Displays colour-coded pie chart slices corresponding to each expense category (e.g., Frozen, Health & Medicine, Drinks).
4. An interactive legend allows toggling category visibility.
5. Aggregated totals are recalculated in real time when filters change.

Scenario 3.2: Adjusting the Date Range

Users can adjust the date range using the date selector located at the top of the dashboard.

Selecting a new range (for example, switching from October 2025 to September–October 2025) automatically refreshes all dashboard data.

The chart, topline metrics, and receipt count are dynamically updated to reflect spending during the selected period.

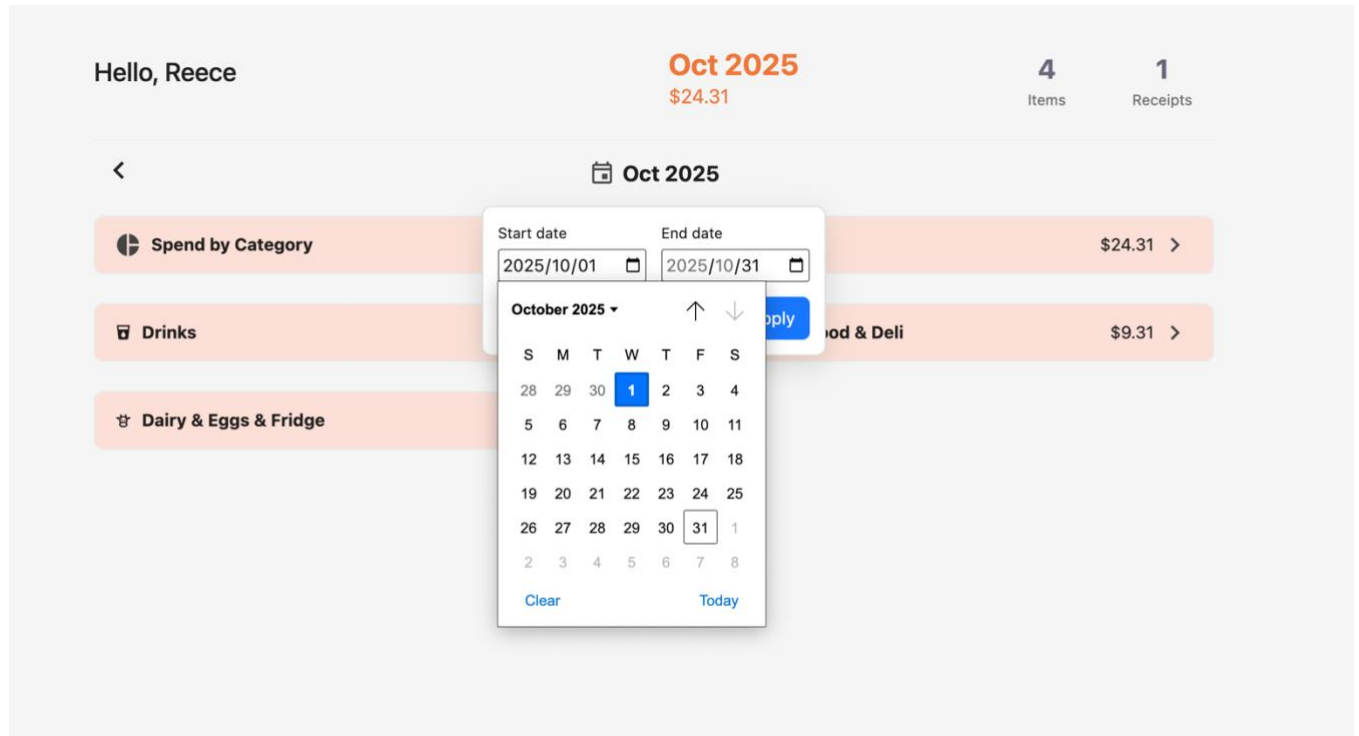


Figure 12 – Date Selector

Feature Highlights:

1. Fetches data dynamically via the same dashboard endpoint with new fromDate and toDate query parameters.
2. Supports arbitrary start and end dates, not limited to full months.
3. Provides immediate visual feedback through a loading state (“Loading...” indicator) while fetching.
4. Uses React state management to propagate updated totals and chart data across components.

Scenario 3.3: Interactive Pie Chart

The CategoryPieCard component enables interactive analysis of spending by category.

Users can click individual legend items to toggle category visibility, effectively including or excluding categories from the chart.

When all categories are hidden, the dashboard displays a special empty-state message with an option to Reset visibility.

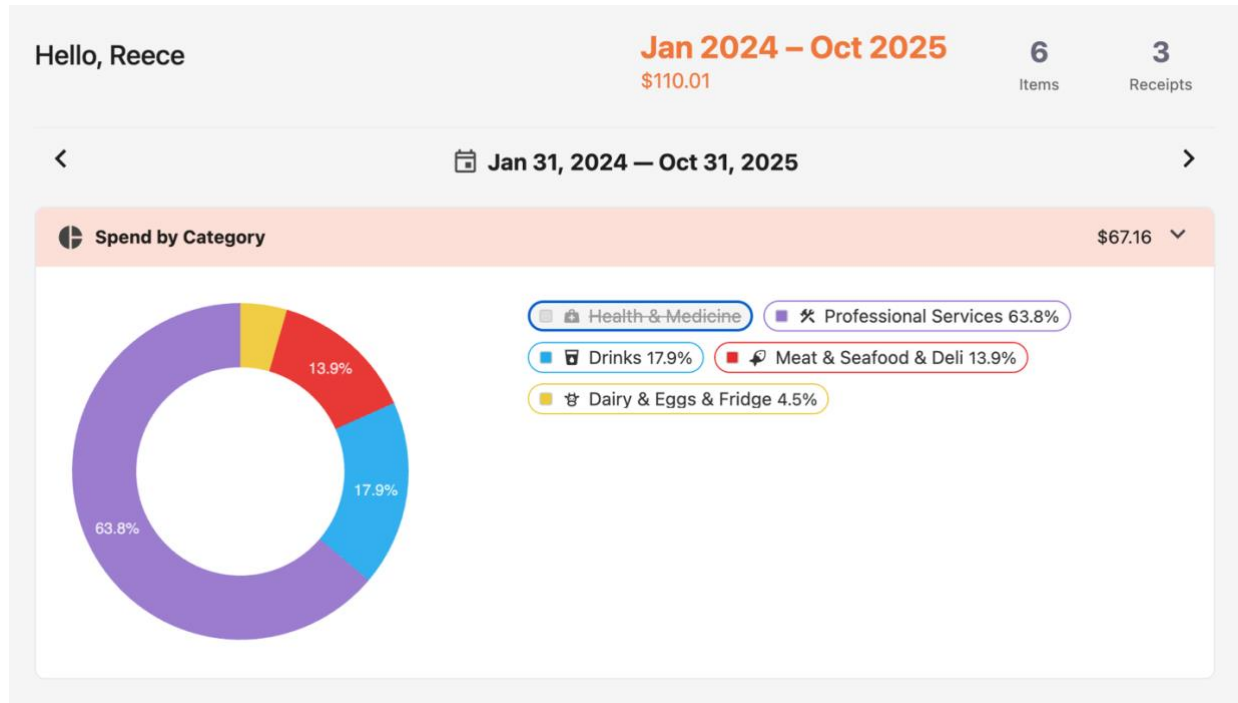


Figure 13 – Toggle pie chart legends

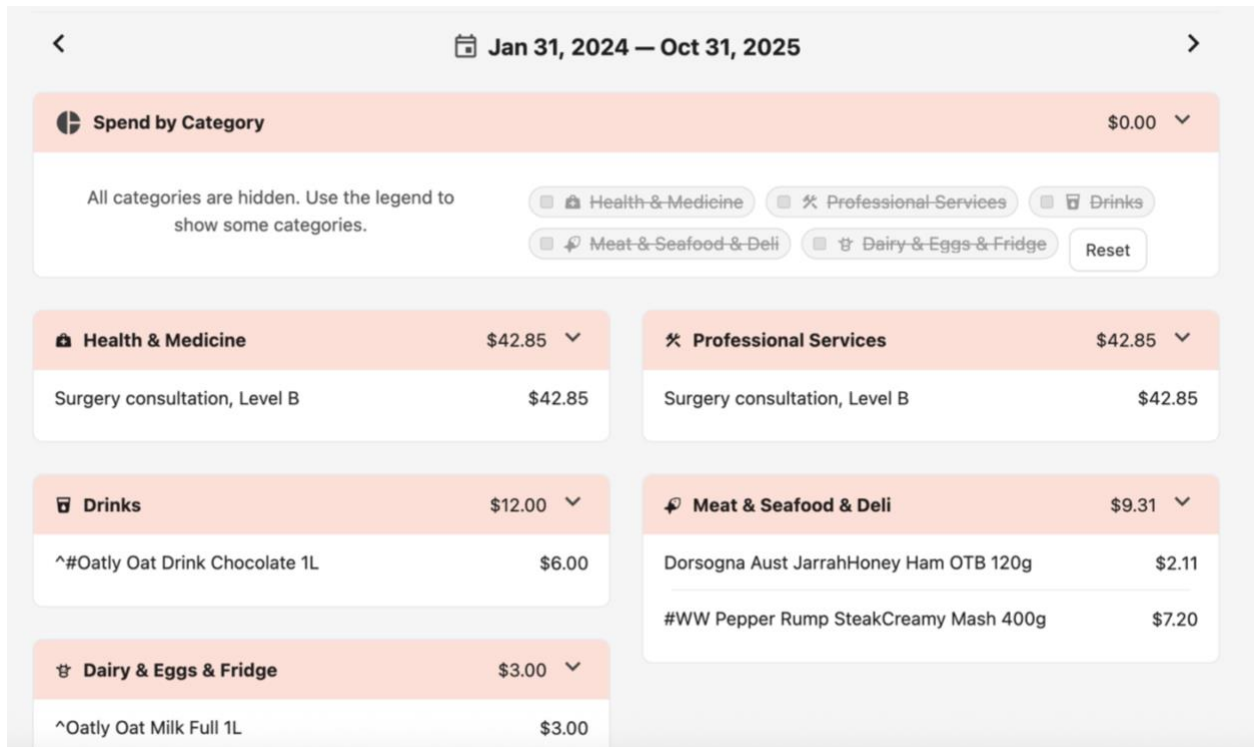


Figure 14 – No category selected

Feature Highlights:

1. Real-time recalculation of visible category percentages.
2. Smooth re-rendering of chart slices using React's dynamic import (SSR disabled for browser-only rendering).
3. Prevents division by zero or NaN totals through safe data normalisation.
4. Predefined colour palette ensures consistent visual distinction between categories.

Scenario 3.4: Empty Data State

If the user has no uploaded receipts within the selected date range, the dashboard still loads normally but displays an empty chart area and a message such as “No receipts found for this period.”

This scenario validates the system’s ability to handle valid queries with empty datasets gracefully.

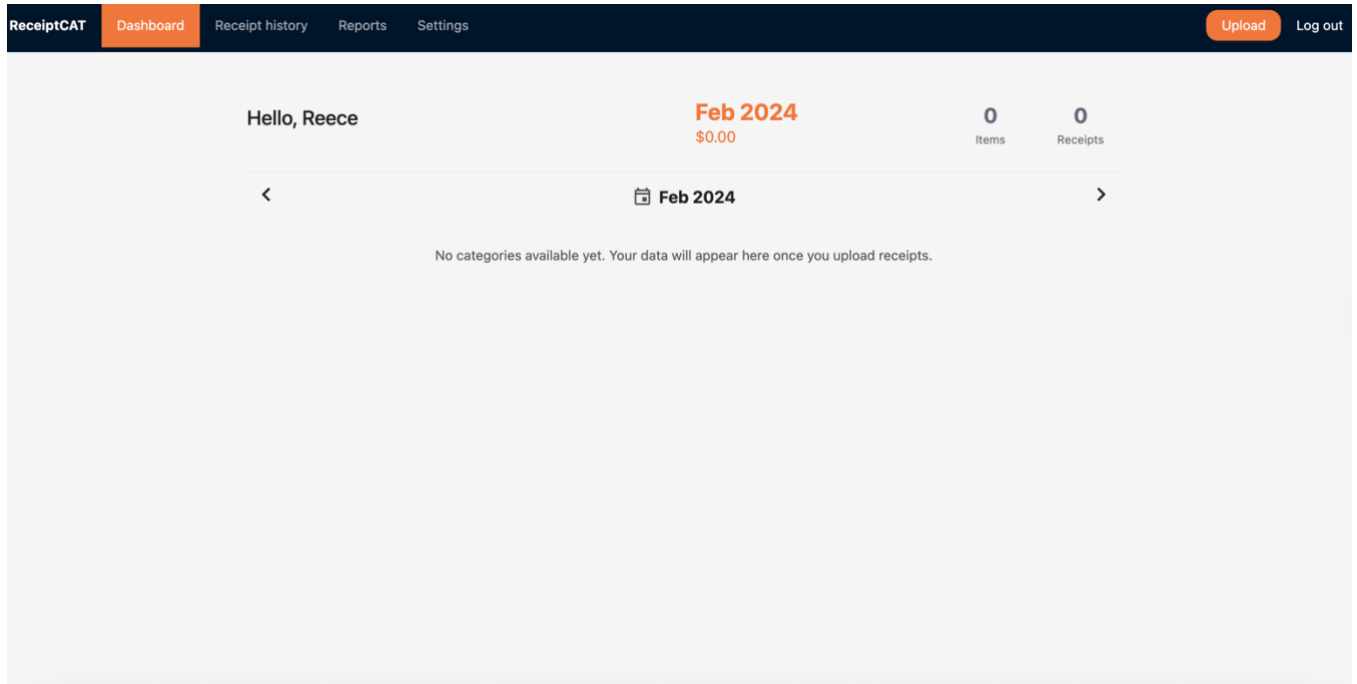


Figure 15 – No data in selected date range

Feature Highlights:

1. Backend returns { receiptsCount: 0, categoryGroups: [] } with HTTP 200 OK, avoiding unnecessary error popups.
2. Frontend renders a user-friendly empty state instead of an error.
3. Maintains a consistent layout and summary section to reassure users that the system is functioning correctly.

4. Receipt History – Viewing Uploaded Receipts

Scenario 4.1: Viewing Past Receipts (Populated State)

After uploading and processing receipts, users can access the **Receipt History** page (/app/history) to view all their past transactions.

Each receipt is summarised as an individual card showing the vendor name, total amount, item count, and purchase date, allowing users to review their spending history at a glance.

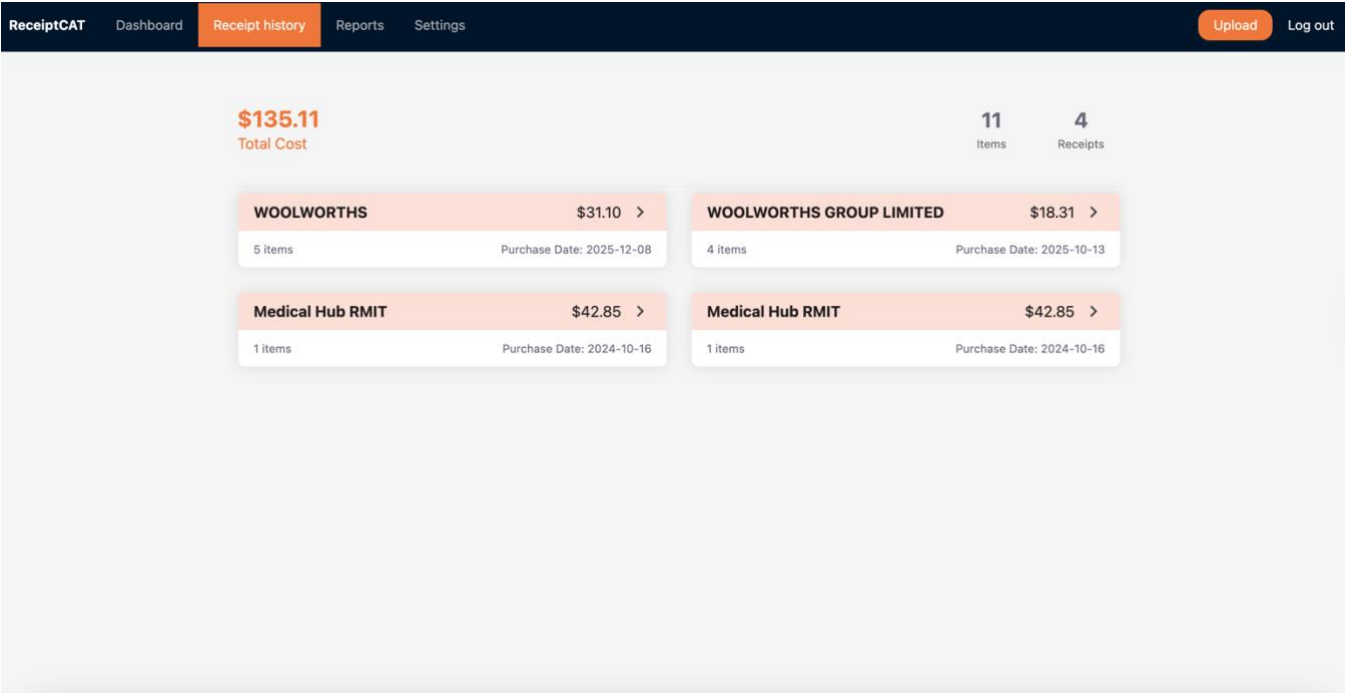


Figure 16 – Receipt History page

Feature Highlights:

1. Displays a summary bar at the top showing Total Cost (sum of all receipts), Total Items (aggregate of all line items), and Receipt Count (number of receipts in history).
2. Each card lists: Vendor name (e.g., Woolworths, Medical Hub RMIT), Item count and purchase date, Total price formatted to two decimal places.
3. Cards are clickable to open detailed receipt views (/app/receipt/{receiptId}).
4. Data is fetched dynamically via the GET /users/{userId}/receipts?fromDate=...&toDate=... endpoint, which returns the total count and individual receipt records.
5. Automatically supports backward compatibility for legacy payloads (arrays without receiptsCount).

Scenario 4.2: No Receipts Found (Empty State)

If the user has not uploaded any receipts or if the selected time range contains no data, the page displays a clear empty-state message instead of an error.

The layout remains consistent, ensuring users understand that the system is functioning correctly, but no receipts are displayed.

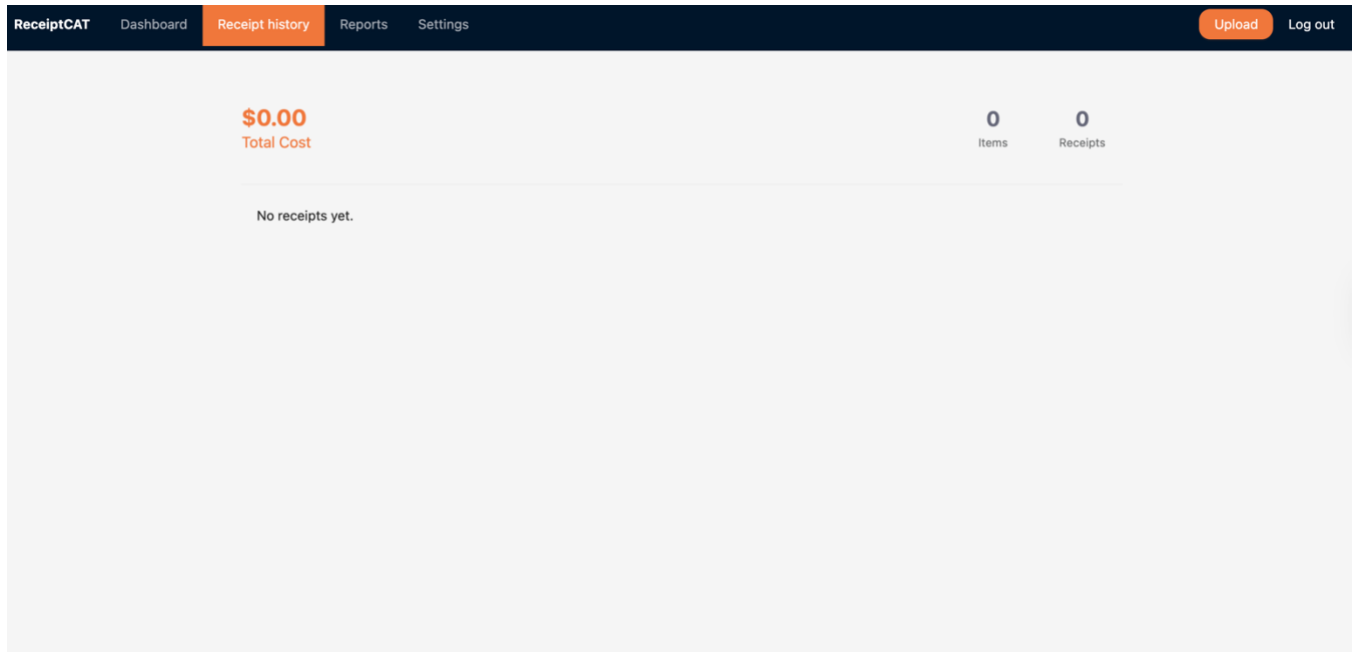


Figure 17 – No data in Receipt History

Feature Highlights:

1. The backend returns a valid HTTP 200 OK with { receiptsCount: 0, categoryGroups: [] }, preventing unnecessary error alerts.
2. The page maintains the same layout and summary section, displaying zeros in total cost and counts.
3. A subtle empty-state message (“No receipts yet”) reassures users that data is simply unavailable, not failed.
4. Consistent UX between Dashboard and Receipt History empty states.

5. Receipt Detail and Editing

Scenario 5.1: Viewing a Receipt – Detail Page (Read-Only Mode)

When a user selects a receipt card from the Receipt History page, they are directed to the Receipt Detail page (/app/receipt/{receiptId}).

This view provides a full breakdown of the selected receipt, displaying both the original receipt image (uploaded by the user) and AI-extracted details such as vendor, date, and individual items.

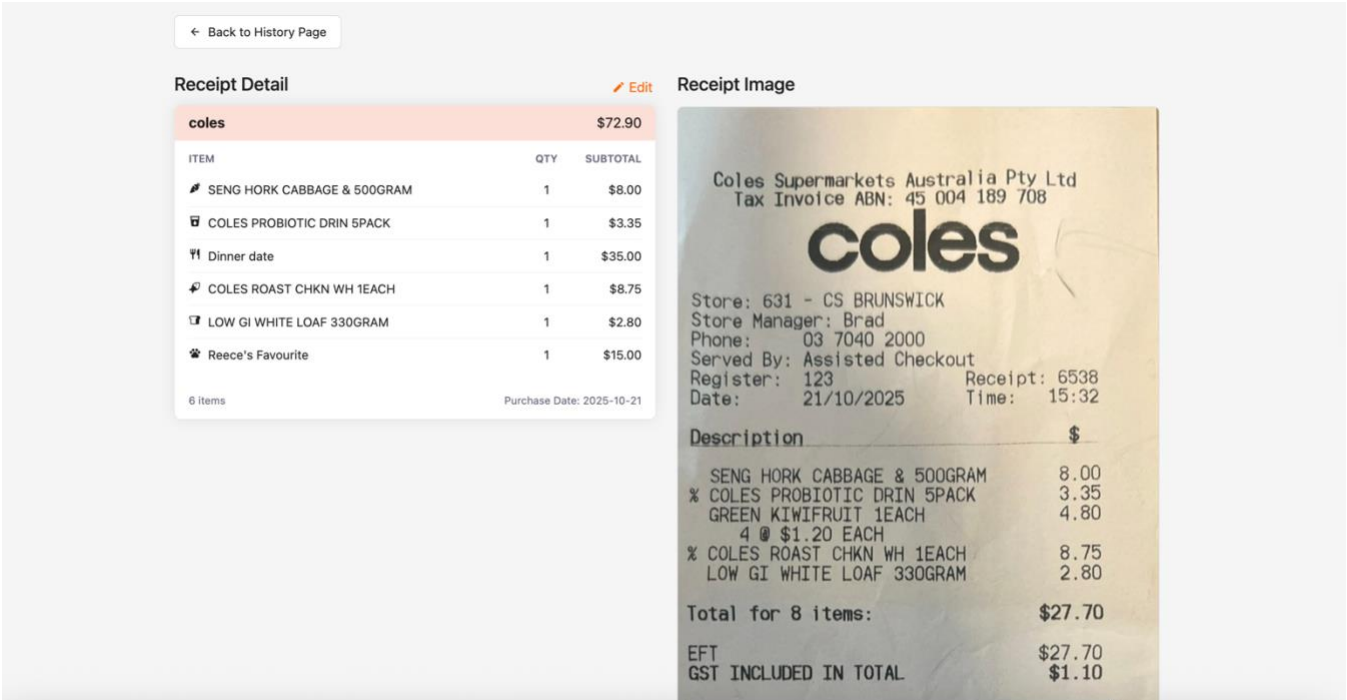


Figure 18 – Receipt Detail page

Feature Highlights:

1. Displays uploaded receipt image on the left and structured purchase data on the right. Shows key metadata fields: Vendor name, Purchase date, Total amount, Number of items
2. The item table includes name, quantity, unit/subtotal price, and AI-assigned category.
3. The data is fetched via: GET /users/{userId}/receipts/{receiptId}, which returns full receipt details, including vendor, date, total, and items[].
4. Provides an Edit button at the top-right corner for transitioning into edit mode when manual correction is needed.

Scenario 5.2: Editing a Receipt – Edit Mode

When the user clicks Edit, the system opens the Edit Mode interface, allowing manual updates to the receipt data.

This mode is essential for correcting errors from AI extraction (e.g., misclassified categories, missing totals, or incorrect item names).

All edits are performed client-side first, with validation applied before saving changes.

The screenshot displays the 'Edit Mode' interface for a receipt. At the top, there is a 'Back to Receipt Detail Page' button. Below it, the 'Edit Mode' title is shown with a 'Save' button on the right. The 'Vendor' field is set to 'coles'. The 'Items' section lists several items with their quantities and prices: 'SENG HORK CABBAGE & 500GRAM' (1, \$8.00), 'COLES PROBIOTIC DRIN 5PACK 1' (\$3.35), 'Dinner date' (1, \$35.00), 'COLES ROAST CHKN WH 1EACH1' (\$8.75), 'LOW GI WHITE LOAF 330GRAM 1' (\$2.80), and 'Reece's Favourite' (1, \$15.00). A modal is open for adding or editing items, with fields for Name, Quantity, Unit Price, Subtotal, and Category. The modal also has 'Cancel' and 'Done' buttons. At the bottom, the 'Gross Total' is \$72.90, and there is a 'Receipt Image' section.

Figure 19 – Receipt Detail edit mode

Feature Highlights:

1. Editable fields: Vendor name (text input), each line item (name, quantity, unit/subtotal price, and category selector). Add, remove, or modify items dynamically.
2. Total recalculates automatically after each edit.
3. The save button remains disabled until all required fields are valid. Unsaved-change protection — attempting to leave the page triggers a confirmation modal (“Leave without saving?”).
4. On save, a PUT request is sent to: `PUT /users/{userId}/receipts/{receiptId}`, with updated data payload `{ vendor, total, items[] }`.
5. Upon success, a toast “Changes saved” appears, and the user is redirected back to the detail page (`/app/receipt/{receiptId}?saved=1`).